DON'T WRITE AUTOMATED TESTS, WRITE EXECUTABLE SPECIFICATIONS

These stories and examples form the basis of the specifications that developers use to build the system. They act as both acceptance criteria, determining when a feature is done, and as guidelines for developers, giving them a clear picture of what needs to be built.

Acceptance criteria give the team a way to objectively judge whether a feature has been implemented correctly. But checking this manually for each code change would be time-consuming and inefficient. It would also slow down feedback, which would in turn slow down the development process. Wherever feasible, teams turn these acceptance criteria into automated acceptance tests or, more precisely, into *executable specifications*.

An executable specification is an automated test that illustrates and verifies how the application delivers a specific business requirement. These automated tests run as part of the build process and run whenever a change is made to the application. In this way, they serve both as acceptance tests, determining which new features are complete, and as regression tests, ensuring that new changes haven't broken any existing features (see figure 1.8).

You can automate an executable specification by writing test code corresponding to each step. BDD tools like Cucumber and JBehave will match the text in each step of your scenario to the appropriate test code.

For example, this is the first step of the scenario in figure 1.8:

```
Given my Current account has a balance of 1000.00
```

You might automate this step in Java using JBehave with code like this:

```
@Given("my $accountType account has a balance of $amount")
public void setupInitialAccount(AccountType type, double amount) {
    Account account = Account.ofType(type).withInitialBalance(amount);
    accountService.create(account);
}
```

❶ The step that this code implements

❷ Call the application code that corresponds to this step

When JBehave runs the scenario, it'll execute each step of the scenario, using basic pattern matching to find the method associated with this step ❶. Once it knows what method to call, it'll extract variables like `type` and `amount` and execute the corresponding application code ❷.

Unlike conventional unit or integration tests, or the automated functional tests many QA teams are used to, executable specifications are expressed in something close to natural language. They use precisely the examples that the users and development team members proposed and refined earlier on, using exactly the same terms and vocabulary. Executable specifications are about communication as much as they are about validation, and the test reports they generate are easily understandable by everyone involved with the project.

These executable specifications also become a single source of truth, providing reference documentation for how features should be implemented. This makes